Redes bayesianas: editor y consultas con eliminación de variables

Memoria técnica

Sebastián Rendon Silva Jesús Rodríguez Uriel Guzman Cardoso

Proyecto de Modelos Probabilistas · 2 de noviembre de 2026

Índice

Introducción, problema y objetivos **2**
Introducción . . . 2
Problema . . . 2
Objetivos . . . 2

Fundamentos de redes bayesianas y eliminación de variables **3**
1. Definición y componentes . . . 3
2. Factorización de la distribución conjunta . . . 3
3. Independencia condicional y d-separación . . . 4
4. Consultas en una red bayesiana . . . 5
5. Eliminación de variables (Variable Elimination) . . . 5

Herramientas y arquitectura **6**
Justificación de las herramientas . . . 6
Arquitectura y componentes . . . 7

Formato de datos **7**
Qué es el formato y para qué se usa . . . 7
Cómo se ve un nodo . . . 7
Qué reglas valida la app . . . 8

Explicación del algoritmo implementado **8**
1. Ubicación y responsabilidades del código . . . 8
2. Representación interna de un factor . . . 8
3. Operaciones sobre factores (`src/factor.js`) . . . 9
4. Algoritmo de eliminación de variables (`BN.infer`) . . . 9
5. Cálculo de referencia: enumeración completa (`BN.enumerate`) . . . 11
6. Complejidad y consideraciones numéricas . . . 11
7. Integración con la interfaz . . . 12

Redes de ejemplo y su verificación **12**
Red de 3 nodos (lluvia, aspersor y pasto) . . . 12
Red de 4 nodos (examen y beca) . . . 12
Red de 5 nodos (contagio, gripe y faltas) . . . 13
Red de 6 nodos (tareas y beca) . . . 13
Red de 10 nodos (fallas de una PC) . . . 13

Pruebas, resultados, limitaciones y posibles mejoras **14**

1. Estrategia de validación 14
2. Pruebas del motor (`tests/test.js`) 15
3. Resultados de los cinco ejemplos 15
4. Prueba de interfaz (`tests/ui-smoke.py`) 16
5. Limitaciones 16
6. Posibles mejoras 17

Conclusiones **17**

Referencias **17**

Contribuciones y declaración de herramientas **18**
Quién hizo qué 18
Declaración de herramientas 18

Introducción, problema y objetivos

Introducción

Una red bayesiana representa con un grafo dirigido acíclico las dependencias entre variables aleatorias discretas y cuantifica cada dependencia con una tabla de probabilidad condicional. Con ella se pueden responder preguntas del tipo «¿cuál es la probabilidad de que ocurra Q sabiendo que se observó e?», que es justamente lo que se necesita en problemas de diagnóstico, como las fallas de un equipo de cómputo o la relación entre estudiar, dormir y aprobar un examen.

Este proyecto entrega una aplicación web que permite construir, visualizar, guardar y consultar redes bayesianas de 3 a 10 nodos, calculando las consultas con el método de **eliminación de variables** (*Variable Elimination*, VE). La aplicación se ejecuta por completo en el navegador, sin servidor.

Problema

Calcular a mano una probabilidad condicional en una red de varios nodos exige multiplicar y sumar tablas de probabilidad con cuidado, y es fácil equivocarse. Se necesita una herramienta que (1) permita definir la red y sus probabilidades, (2) valide que la red sea correcta, (3) calcule las consultas con evidencia y (4) permita comprobar que el resultado es el esperado.

Objetivos

Objetivo general. Desarrollar una aplicación web, sin servidor, para construir y consultar redes bayesianas mediante eliminación de variables.

Objetivos específicos.

- Representar redes de 3 a 10 nodos con estados de cualquier tamaño (al menos dos) y tablas de probabilidad condicional, con un formato de archivo JSON documentado.
- Validar las redes: grafo acíclico, identificadores únicos, padres existentes y filas de cada tabla que sumen 1.
- Implementar la eliminación de variables, con poda de nodos irrelevantes, e indicar el orden y los pasos de eliminación.
- Verificar cada resultado contra un cálculo independiente por enumeración completa y contra cálculos hechos a mano.
- Ofrecer una interfaz para dibujar la red, editar nodos, consultar con evidencia, guardar y abrir archivos, imprimir y exportar el grafo como imagen.
- Incluir cinco redes de ejemplo, de 3, 4, 5, 6 y 10 nodos, con su consulta resuelta y verificada.

Fundamentos de redes bayesianas y eliminación de variables

> Esta sección forma parte de la memoria técnica del proyecto «Redes bayesianas: editor y consultas con eliminación de variables». Se ubica después del planteamiento del problema y los objetivos, y antes de la justificación de las herramientas y la arquitectura del software. Los conceptos aquí expuestos son la base de la sección «Explicación del algoritmo implementado», donde se describe su traslado a código. El formato de archivo de una red se documenta en la sección «Formato de datos» y la interfaz gráfica en «Módulo de interfaz y pruebas».

1. Definición y componentes

Una **red bayesiana** es un modelo gráfico probabilístico que representa la distribución de probabilidad conjunta de un conjunto de variables aleatorias discretas mediante dos elementos complementarios:

- **Parte cualitativa (estructura).** Un **grafo dirigido acíclico (DAG)** $G = (V, E)$, donde cada vértice $v \in V$ es una variable aleatoria y cada arista dirigida $u \to v$ expresa que u influye directamente sobre v. Se dice que u es **padre** de v y que v es **hijo** de u.
- **Parte cuantitativa (parámetros).** Para cada variable X, una **tabla de probabilidad condicional (CPT)** que especifica $P(X = x \mid Pa(X) = pa)$ para cada estado x de X y cada combinación de estados pa de sus padres $Pa(X)$.

Formalmente, una red bayesiana sobre las variables $X_1, \ldots, X_n$ es un par (G, θ) donde G es un DAG y $\theta = \{ P(X_i \mid Pa(X_i)) \}$ contiene la condicional local de cada variable dados sus padres. En este proyecto las variables son discretas con al menos dos estados cada una, la red debe tener entre 3 y 10 nodos, y las filas de cada CPT deben sumar 1. Las reglas exactas de validación se detallan en la sección «Formato de datos».

La restricción de que el grafo sea **acíclico** no es un detalle menor: garantiza que existe un orden topológico de las variables (padres antes que hijos) y, por tanto, que la factorización de la siguiente sección está bien definida. Si existiera un ciclo, una variable terminaría dependiendo de sí misma a través del ciclo y no habría una distribución conjunta consistente.

2. Factorización de la distribución conjunta

La propiedad central de una red bayesiana es la **factorización**: la distribución conjunta se descompone como el producto de las condicionales locales,

$$P(X_1, \ldots, X_n) = \prod_i P(X_i \mid Pa(X_i)).$$

Esta identidad se obtiene aplicando la regla de la cadena en un orden topológico y usando la **propiedad local de Markov**: cada variable es condicionalmente independiente de sus no descendientes dados sus padres. La red no afirma que las variables sean arbitrariamente independientes entre sí; afirma que, dados sus padres, cada variable queda determinada sin necesidad de mirar más nodos.

La factorización es lo que hace viable la inferencia. Si se almacenara la conjunta completa de n variables binarias se necesitarían $2^n - 1$ números independientes (por ejemplo, 1023 para 10 variables). En una red bayesiana se almacenan en su lugar las CPT locales: para un nodo con k estados y padres cuyos números de estados son $c_1, \ldots, c_m$, su CPT tiene $k \cdot c_1 \cdot \ldots \cdot c_m$ entradas (con $(k-1) \cdot c_1 \cdot \ldots \cdot c_m$ grados de libertad). En redes poco densas esta cantidad es mucho menor que $2^n - 1$.

3. Independencia condicional y d-separación

3.1. Independencia y dependencia condicional

Dos variables X e Y son **independientes** si $P(X, Y) = P(X) \cdot P(Y)$. Son **condicionalmente independientes dada Z** (se escribe $X \perp Y \mid Z$) si

$$P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z).$$

En una red bayesiana, las aristas no son meras líneas: según su orientación relativa, determinan si la información puede «fluir» entre dos variables a través de un camino, y si observar (condicionar sobre) una variable intermedia bloquea o, por el contrario, abre ese flujo. Este comportamiento se formaliza con la **d-separación**.

3.2. Las tres estructuras locales

Cualquier camino no dirigido entre dos variables está compuesto por triples consecutivos de tres tipos:

a) Cadena (head-to-tail): $X \to Z \to Y$.

Z es causa de Y y efecto de X. En general X e Y son dependientes entre sí (el estado de X influye en Y a través de Z), pero si se observa Z, el camino queda **bloqueado** y $X \perp Y \mid Z$. Un ejemplo del proyecto es la cadena `Contagio → Gripe → Fiebre` (ejemplo 3): conocer `Contagio` informa sobre `Fiebre`; pero si ya se conoce `Gripe`, el valor de `Contagio` deja de aportar información sobre `Fiebre`.

b) Bifurcación o causa común (tail-to-tail): $X \leftarrow Z \to Y$.

Z es causa común de X e Y, que son efectos correlacionados por compartir esa causa: X e Y son dependientes en general, pero observar Z **bloquea** el camino y vuelve a X e Y condicionalmente independientes: $X \perp Y \mid Z$.

c) Colisionador o estructura en V (head-to-head): $X \to Z \leftarrow Y$.

Aquí Z recibe influencia de dos causas X e Y que, a priori, son **independientes**: $X \perp Y$. Sin embargo, observar Z (o uno de sus descendientes) **abre** el camino y hace que X e Y se vuelvan dependientes. Este fenómeno se conoce como *explaining away*: explicar el efecto a través de una causa reduce la necesidad de la otra. En el ejemplo 2 (examen y beca), `Aprueba` es colisionador de `Estudia` y `Duerme`: a priori, haber estudiado y haber dormido son independientes; pero condicionando sobre `Aprueba = sí`, ambas variables se correlacionan (sabiendo que el estudiante aprobó, si se entera de que durmió poco, aumenta la probabilidad de que haya estudiado mucho).

3.3. Definición de d-separación

Sea S un conjunto de variables observadas. Un **camino activo** entre X e Y dado S es aquel en el que todo triple consecutivo está «abierto»:

- en una cadena $\to Z \to$ o $\leftarrow Z \leftarrow$, y en una bifurcación $\leftarrow Z \to$, el nodo intermedio Z **no** está en S;
- en un colisionador $\to Z \leftarrow$, el nodo intermedio Z **o alguno de sus descendientes** está en S.

Se dice que S **d-separa** a X de Y (o que X e Y son **d-separadas por S**) si **no existe ningún camino activo** entre ellas dado S. La definición se extiende a conjuntos: S d-separa los conjuntos A y B si d-separa todo par $X \in A, Y \in B$.

El resultado fundamental (teorema de **corrección/solidez**) es:

> Si S d-separa a X de Y en el grafo G, entonces $X \perp Y \mid S$ se cumple en toda distribución que factorice según G.

La implicación recíproca (que toda independencia observada en los datos provenga de una d-separación del grafo) solo se cumple para distribuciones **fieles** al grafo; es una propiedad usual pero no universal. Por ello, la d-separación es la herramienta gráfica para *leer* las independencias que la red garantiza.

3.4. Cobertura de Markov

Como consecuencia de la d-separación, cada variable X es condicionalmente independiente de todas las demás variables de la red dados su **cobija de Markov** (*Markov blanket*): sus padres, sus hijos y los otros padres de sus hijos. Este es el conjunto mínimo que contiene toda la información necesaria para predecir X.

4. Consultas en una red bayesiana

Sobre una red construida pueden formularse tres tipos de consultas:

- **Marginal o a priori:** P(X), la probabilidad de un nodo sin condicionar.
- **Conjunta:** P(X, Y, …), la probabilidad de una combinación de estados.
- **Condicional o a posteriori:** P(Q | E = e), la distribución de una o más variables de consulta Q dado que las variables de evidencia E toman los valores e.

En general, responder una consulta requiere **sumar (marginalizar) sobre todas las variables ocultas** —aquellas que no son de consulta ni de evidencia—:

$$P(Q \mid E = e) = \Sigma_h \, P(Q, H = h \mid E = e) \propto \Sigma_h \prod_i P(X_i \mid Pa(X_i)),$$

donde H son las variables ocultas. La suma sobre todas las combinaciones de estados de H es exacta pero costosa: crece con el producto de los tamaños de los dominios. El método de **eliminación de variables** evita calcular la conjunta completa explotando la factorización.

5. Eliminación de variables (Variable Elimination)

5.1. Factores

Un **factor** es una función φ(alcance) que asigna un número real no negativo a cada asignación de estados de un conjunto de variables llamado **alcance**. Las CPT de la red son factores: la del nodo X es $\varphi(Pa(X) \cup \{X\}) = P(X \mid Pa(X))$. El producto de factores y la suma sobre variables producen nuevos factores.

5.2. Operaciones sobre factores

1. **Producto punto a punto.** $(\varphi \cdot \psi)(a) = \varphi(a) \cdot \psi(a)$. El alcance del resultado es la unión de los alcances.
2. **Suma fuera (marginalización).** $(\Sigma_Z \varphi)(a) = \Sigma_{z \in dom(Z)} \varphi(a, Z = z)$. La variable Z desaparece del alcance.
3. **Restricción (evidencia).** Fijar Z = z y eliminar Z del alcance, conservando solo las entradas compatibles con la evidencia.
4. **Normalización.** Dividir todas las entradas por su suma total para obtener una distribución que suma 1.

La identidad que hace eficiente el método es la **propiedad distributiva**:

$$\Sigma_Z (\varphi_1 \cdot \ldots \cdot \varphi_k) = (\text{producto de los } \varphi_i \text{ que no mencionan a Z}) \cdot \Sigma_Z (\text{producto de los } \varphi_i \text{ que mencio}$$

«Empujar» la suma hacia adentro reduce el tamaño de las tablas que se multiplican, que es exactamente lo que explota la eliminación de variables.

5.3. Algoritmo

Dada una red con factores locales Φ, variables de consulta Q y evidencia E = e:

```
VE(Φ, Q, E):
  1. F ← { factor local P(X | Pa(X)) por cada nodo X }
  2. F ← { restringir(f, E) para cada f en F }          # incorporar la evidencia
  3. H ← variables que no están en Q ni en E            # variables ocultas
  4. π ← orden de eliminación de H
  5. para cada Z en π:
       F' ← { f en F | Z está en el alcance de f }
       g ← Σ_Z ( ∏_{f en F'} f )                        # multiplicar y sumar fuera
       F ← (F \ F') ∪ { g }
  6. h ← ∏_{f en F} f                                   # factor final sobre Q
  7. devolver h / Σ_Q h                                 # normalizar → P(Q | E = e)
```

El resultado es exacto y no depende del orden de eliminación: cualquier orden produce la misma distribución. Lo que sí cambia con el orden es el **coste**: el tamaño de los factores intermedios. El coste de eliminar Z está dominado por el tamaño del factor producto (el producto de los tamaños de dominio de las variables de su alcance), de modo que el coste total depende de la elección de π y, en el peor caso, es exponencial en la **anchura de árbol** (*treewidth*) de la red. La sección «Explicación del algoritmo implementado» describe el criterio concreto (heurística *greedy*) que usa la aplicación para elegir el orden.

5.4. Ejemplo numérico ilustrativo

Considérese la cadena `A → B → C` con variables binarias y parámetros:

`P(A = V) = 0.6`	`P(A = F) = 0.4`
`P(B = V | A = V) = 0.8`	`P(B = V | A = F) = 0.3`
`P(C = V | B = V) = 0.9`	`P(C = V | B = F) = 0.2`

Para la consulta `P(A = V | C = V)`:

1. **Factores:** `f_A(A)`, `f_B(A, B)`, `f_C(B, C)`.
2. **Evidencia `C = V`:** se restringe `f_C` a `f_C(B) = [0.9, 0.2]` para `B = V, F`.
3. **Eliminar `B`:** `g(A) = Σ_B f_B(A, B) · f_C(B)`:
   - `g(A = V) = 0.8 · 0.9 + 0.2 · 0.2 = 0.72 + 0.04 = 0.76`
   - `g(A = F) = 0.3 · 0.9 + 0.7 · 0.2 = 0.27 + 0.14 = 0.41`
4. **Multiplicar por `f_A`:** `h(A = V) = 0.6 · 0.76 = 0.456`, `h(A = F) = 0.4 · 0.41 = 0.164`.
5. **Normalizar:** `P(C = V) = 0.456 + 0.164 = 0.62`, de donde
   - `P(A = V | C = V) = 0.456 / 0.62 = 0.7355`
   - `P(A = F | C = V) = 0.164 / 0.62 = 0.2645`.

Obsérvese que en el paso 3 la suma se aplicó solo al producto de los dos factores que mencionan a `B`, y `f_A` se mantuvo aparte hasta el final: esa es la esencia del ahorro del método.

La sección «Explicación del algoritmo implementado» muestra cómo cada una de estas operaciones se materializa en los módulos `src/factor.js` y `src/infer.js`, y la sección «Pruebas, resultados, limitaciones y posibles mejoras» recoge la verificación numérica de estos resultados.

Herramientas y arquitectura

Justificación de las herramientas

- **HTML, CSS y JavaScript sin servidor.** La aplicación se abre con doble clic en `index.html` y también funciona publicada en GitHub Pages; no hay nada que instalar ni compilar. El mismo

código del motor funciona en el navegador y en Node.js, lo que permite probarlo desde la línea de comandos.
- **Cytoscape.js** (licencia MIT, incluida en `vendor/`) para dibujar el grafo, mover los nodos y exportar la imagen, sin depender de una conexión a internet.
- **Node.js** (versión 18 o superior) para las pruebas automáticas, sin dependencias adicionales.
- **Playwright** (Python) para una prueba opcional de la interfaz en un navegador automatizado.
- **Git y GitHub** para el control de versiones, con ramas por integrante y *pull requests*, y **GitHub Pages** para la versión en línea.

Arquitectura y componentes

Componente	Archivo	Responsabilidad
Factores	`src/factor.js`	Producto, suma fuera, restricción por evidencia y normalización
Inferencia	`src/infer.js`	Eliminación de variables, poda, enumeración de referencia y comparación
Validación	`src/validate.js`	Validación de redes, carga y guardado en JSON
Interfaz	`src/app.js`, `index.html`, `css/estilos.css`	Grafo, consultas, resultados, edición de nodos, reporte
Ejemplos	`ejemplos/`, `src/ejemplos.generated.js`	Cinco redes en JSON y su versión empaquetada para el navegador
Pruebas	`tests/`	Pruebas del motor y prueba de la interfaz
Documentación	`docs/`	Formato de archivo, verificaciones a mano y evidencias

La interfaz usa el motor únicamente a través de funciones públicas (`BN.validateNetwork`, `BN.infer`, `BN.enumerate`, `BN.compareResults`), de modo que el motor puede probarse sin interfaz.

Formato de datos

Qué es el formato y para qué se usa

Una red bayesiana se almacena en un archivo de texto con formato JSON (codificación UTF-8, extensión `.json`). Ese mismo formato sirve tanto para que el usuario guarde su red y la vuelva a abrir después, como para importar redes ya hechas y para los ejemplos que trae la aplicación.

Cómo se ve un nodo

El archivo tiene un objeto principal con la versión del formato, un nombre para mostrar y la lista de nodos. `version` y `name` son opcionales; lo único obligatorio es `nodes`. Cada nodo guarda su identificador único, la lista de valores que puede tomar, los identificadores de sus padres (vacía si no tiene) y su tabla de probabilidades condicionales. Por ejemplo, el nodo `Aspersor` tiene dos estados (`encendido` / `apagado`) y un solo padre (`Lluvia`), que también tiene dos estados (`sí` / `no`); por eso su tabla necesita exactamente dos filas: una para cuando `Lluvia=sí` y otra para cuando

`Lluvia=no`. En cada fila se indica el estado de los padres y la probabilidad de cada estado del nodo, de modo que las probabilidades de una misma fila siempre suman 1. Opcionalmente se puede guardar también la posición de cada nodo en pantalla, pero eso es solo para dibujar el grafo y no afecta a los cálculos.

Qué reglas valida la app

Antes de aceptar un archivo, la aplicación comprueba varias cosas: que la red tenga entre 3 y 10 nodos; que los identificadores no estén vacíos ni repetidos y que los padres existan y no formen ciclos (es decir, que el grafo sea dirigido acíclico); que cada nodo tenga al menos dos estados distintos; y que la tabla de probabilidades sea coherente, con una fila por cada combinación de estados de los padres, probabilidades entre 0 y 1 y filas que sumen 1 con una tolerancia de 10^{-6}. Si algo falla, la app rechaza la red y avisa del problema. No se admiten ciclos porque, en una red bayesiana, la probabilidad de cada nodo depende solo de sus padres; si el grafo tuviera un ciclo, un nodo terminaría dependiendo de sí mismo a través del ciclo y la tabla de probabilidades dejaría de estar bien definida (no habría un orden para calcularla ni una distribución conjunta válida).

Explicación del algoritmo implementado

> Esta sección forma parte de la memoria técnica del proyecto «Redes bayesianas: editor y consultas con eliminación de variables». Presupone los conceptos de la sección «Fundamentos de redes bayesianas y eliminación de variables» (factores, producto, suma fuera, restricción, normalización) y describe cómo se trasladan al código real. La validación de redes y el formato de archivo se documentan en la sección «Formato de datos»; la forma en que la interfaz invoca este motor se describe en «Módulo de interfaz y pruebas».

1. Ubicación y responsabilidades del código

El motor de inferencia vive en dos módulos que funcionan tanto en el navegador (objeto global `BN`) como en Node.js:

- **`src/factor.js`** — define la estructura de un factor y sus operaciones (creación desde una CPT, producto, suma fuera, restricción por evidencia y normalización).
- **`src/infer.js`** — implementa el algoritmo de eliminación de variables (`BN.infer`) y un cálculo de referencia independiente por enumeración completa (`BN.enumerate`), además de la comparación entre ambos (`BN.compareResults`).
- **`src/validate.js`** — validación de redes, carga y guardado (`BN.validateNetwork`, `BN.loadNetwork`, `BN.serializeNetwork`), usado al inicio de la inferencia para rechazar redes mal formadas.

2. Representación interna de un factor

Un factor es una tabla sobre un conjunto ordenado de variables:

```
{ vars: ["A", "B"],           // alcance, ordenado
  states: { A: [..], B: [..] }, // estados de cada variable
  card: [nA, nB],             // cardinalidades
  table: [ ... ] }            // valores numéricos
```

La `table` se almacena en orden **row-major**: la **última** variable cambia más rápido. El índice lineal de una asignación es

$$\text{índice} = \Sigma_i \; \text{idx}_i \cdot \text{stride}_i, \quad \text{con stride de la última variable} = 1$$

y $\text{stride}_i = \prod_{j > i} \text{card}_j$. Esta convención permite recorrer las tablas con aritmética de índices sin diccionarios, lo que mantiene las operaciones rápidas y deterministas.

3. Operaciones sobre factores (`src/factor.js`)

`fromNode(net, node)` construye el factor local `P(X | Pa(X))` a partir de la CPT. El alcance es `[ ...padres, X ]` y cada entrada se llena con `row.probs[s]` para la asignación `{ ...row.given, X: s }`:

```
function fromNode(net, node) {
  const statesMap = {};
  net.nodes.forEach((n) => (statesMap[n.id] = n.states));
  const f = make(node.parents.concat([node.id]), statesMap);
  node.cpt.forEach((row) => {
    node.states.forEach((s) => {
      const a = Object.assign({}, row.given);
      a[node.id] = s;
      f.table[indexOf(f, a)] = row.probs[s];
    });
  });
  return f;
}
```

`multiply(f, g)` crea un factor sobre la unión de los alcances y asigna a cada celda el producto de las celdas correspondientes de `f` y `g`. Recorre la tabla de salida con un contador mixto (el mismo esquema de *strides*) y, para las variables ausentes en un factor, su *stride* se trata como 0 (la celda no cambia).

`sumOut(f, v)` marginaliza sobre `v`: suma, para cada combinación de las demás variables, todas las entradas que comparten esa combinación y difieren solo en el estado de `v`. El resultado pierde la variable `v` de su alcance.

`restrict(f, v, state)` incorpora evidencia: conserva únicamente las entradas cuyo índice para `v` coincide con el estado indicado y elimina `v` del alcance. Es la materialización de fijar `v = state`.

`normalize(f)` divide todas las entradas por su suma total `t = Σ table` y devuelve `{ factor, total }`. El valor `total` es justamente `P(evidencia)`, que la interfaz muestra y que las pruebas usan para verificar los cálculos a mano.

4. Algoritmo de eliminación de variables (`BN.infer`)

`BN.infer(net, queryVars, evidence, opts)` sigue el esquema general descrito en la sección de fundamentos, con dos añadidos prácticos: una **poda** previa de nodos irrelevantes y una **heurística greedy** para el orden de eliminación. Sus pasos son:

4.1. Validación de la red y de la consulta

Primero se llama a `BN.validateNetwork(net, { enforceSize: false })`: se comprueban identificadores, estados, padres, aciclicidad y coherencia de las CPT, pero sin imponer el límite de 3–10 nodos (ese límite ya se aplica al cargar/guardar; la inferencia puede operar sobre cualquier red estructuralmente válida). Después `checkQuery` valida la consulta:

- hay al menos una variable de consulta y no está repetida;
- toda variable de consulta y de evidencia existe en la red;
- ninguna variable es a la vez consulta y evidencia;
- cada estado de evidencia es un estado válido de su variable.

4.2. Poda de nodos irrelevantes

Se calcula el conjunto de **ancestros** (incluida la propia variable) de las variables de consulta y de evidencia:

```
const relevant = ancestorsOf(byId, queryVars.concat(Object.keys(evidence)));
```

Los nodos que no son ancestros de Q ni de E se descartan. Esto es correcto porque la CPT de un nodo irrelevante suma 1 en cada fila, de modo que al marginalizarlo se anula su contribución y no altera el resultado final. La poda puede desactivarse con `opts.prune = false` (las pruebas verifican que ambos caminos coinciden).

4.3. Construcción de factores e incorporación de la evidencia

Para cada nodo relevante se crea su factor con `fromNode` y, a continuación, se aplica `restrict` por cada variable de evidencia:

```
let factors = net.nodes
  .filter((n) => relevant.has(n.id))
  .map((n) => {
    let f = Fa.fromNode(net, n);
    Object.keys(evidence).forEach((e) => { f = Fa.restrict(f, e, evidence[e]); });
    return f;
  });
```

4.4. Orden de eliminación

Las variables ocultas son las relevantes que no son de consulta ni de evidencia. Si el usuario indica un orden (`opts.order`), las variables indicadas se respetan en ese orden. Las restantes se eligen con una **heurística greedy de mínimo tamaño**: en cada paso se elimina la variable que produce el factor resultante más pequeño (el producto de las cardinalidades de las variables de su alcance tras unir todos los factores que la contienen):

```
function greedyPick() {
  let best = null, bestCost = Infinity;
  remaining.slice().sort().forEach((h) => {
    const vars = new Set();
    factors.filter((f) => f.vars.includes(h)).forEach((f) => f.vars.forEach((x) => vars.add
    vars.delete(h);
    let cost = 1;
    vars.forEach((x) => (cost *= byId[x].states.length));
    if (cost < bestCost) { bestCost = cost; best = h; }
  });
  return best;
}
```

Este criterio aproxima el orden óptimo sin garantizarlo: es una heurística de coste bajo, razonable para redes de hasta 10 nodos.

4.5. Bucle de eliminación

Para cada variable oculta, en el orden elegido, se multiplican todos los factores que la contienen y se suma fuera sobre ella; el factor resultante sustituye a los originales. Cada paso se registra en una **traza**:

```
while (remaining.length) {
  const h = forced.length ? forced.shift() : greedyPick();
  remaining.splice(remaining.indexOf(h), 1);
```

```
  order.push(h);
  const involved = factors.filter((f) => f.vars.includes(h));
  const rest = factors.filter((f) => !f.vars.includes(h));
  if (involved.length === 0) continue;
  const product = involved.reduce((a, b) => Fa.multiply(a, b));
  const summed = Fa.sumOut(product, h);
  factors = rest.concat([summed]);
  trace.push({ eliminated: h, multiplied: involved.map((f) => f.vars.slice()),
               productVars: product.vars.slice(), resultVars: summed.vars.slice() });
}
```

4.6. Normalización y salida

Al terminar, se multiplican los factores restantes (que solo involucran variables de consulta) y se normaliza:

```
const finalF = factors.reduce((a, b) => Fa.multiply(a, b));
const norm = Fa.normalize(finalF);
if (!(norm.total > 0)) throw new Error('La evidencia tiene probabilidad 0: no se puede cond
```

Si `total` es 0, la evidencia es imposible y la consulta no está definida; la aplicación lo informa como error en lugar de dividir entre cero.

El resultado devuelto contiene:

- `query` y `evidence` (la consulta y la evidencia utilizadas);
- `pEvidence` — la probabilidad de la evidencia, $P(E = e)$;
- `joint` — la distribución conjunta $P(Q \mid e)$ para **todas** las combinaciones de estados de las variables de consulta;
- `marginals` — la marginal de cada variable consultada (suma de `joint` sobre las demás);
- `order`, `trace` y `prunedNodes` — el orden de eliminación, los pasos intermedios y los nodos podados, que la interfaz muestra para hacer el proceso inspeccionable.

5. Cálculo de referencia: enumeración completa (`BN.enumerate`)

Para poder verificar el resultado sin recurrir al propio VE, se implementó un segundo método **independiente**: `BN.enumerate(net, queryVars, evidence)` recorre recursivamente **todas** las combinaciones de estados de la red (restringiendo las variables de evidencia a su valor observado), acumula la probabilidad conjunta $\prod P(X_i \mid Pa(X_i))$ de cada combinación y luego normaliza. Es el «cálculo de fuerza bruta» de la fórmula de factorización, sin usar `factor.js`. Al ser conceptualmente distinto de VE, su coincidencia numérica es una evidencia fuerte de corrección.

`BN.compareResults(a, b)` devuelve la máxima diferencia absoluta entre dos resultados (en `pEvidence` y en cada celda de `joint`). La interfaz muestra esta diferencia cuando el usuario activa la casilla «comparar con enumeración», y la suite de pruebas la usa como criterio de aceptación.

6. Complejidad y consideraciones numéricas

El coste de eliminar una variable es proporcional al tamaño del factor producto de los factores que la mencionan; en total, VE es $O(\sum \text{tamaño de los factores intermedios})$. El tamaño de un factor es el producto de las cardinalidades de su alcance, por lo que el coste depende del orden de eliminación y, en el peor caso, crece de forma exponencial con la **anchura de árbol** de la red. La enumeración completa, en cambio, es siempre $O(\prod \text{card}(X_i))$, es decir, exponencial en el número total de variables; VE solo se degrada hasta ese extremo cuando la red es muy densa.

La implementación usa aritmética de punto flotante de doble precisión. La normalización garantiza que la distribución resultante sume 1, y las diferencias observadas frente a la

enumeración son del orden de 10^{-15} (véase la sección «Pruebas, resultados, limitaciones y posibles mejoras»), consistentes con el error de redondeo acumulado, no con un error del método.

7. Integración con la interfaz

La función `run()` de `src/app.js` lee las variables marcadas como consulta y la evidencia seleccionada en los selectores, ejecuta `BN.infer(net, q, e)` y, si está activada la comparación, `BN.compareResults(res, BN.enumerate(net, q, e))`. `renderResult()` muestra las marginales (con barras), la conjunta cuando hay varias variables de consulta, `P(evidencia)`, la verificación frente a la enumeración y el detalle desplegable «Orden y pasos de eliminación» construido a partir de `res.trace`. `renderReport()` compone el reporte imprimible con la estructura de la red, las CPT y el resultado. Los detalles de construcción de esa interfaz se describen en la sección «Módulo de interfaz y pruebas».

Redes de ejemplo y su verificación

Red de 3 nodos (lluvia, aspersor y pasto)

Qué modela: Esta es la red más pequeña del proyecto y representa las causas de que el pasto de un jardín esté mojado. Tiene 3 nodos binarios. En la raíz está `Lluvia`, de la cual dependen tanto el `Aspersor` como el estado del `Pasto`. Además, el `Aspersor` también influye en el `Pasto`, por lo que este nodo final tiene dos padres.

La consulta: La consulta resuelta fue **P(Lluvia = sí | Pasto = mojado)**. Es decir, cuál es la probabilidad de que haya llovido sabiendo que el pasto está mojado. El resultado obtenido fue **0.357688** (y P(Lluvia = no | Pasto = mojado) = 0.642312).

Cómo lo verifiqué: Para verificarlo, apliqué el método de eliminación de variables. En esta red no hay nodos que podar, ya que los tres son relevantes (`Lluvia` es la consulta, `Pasto` es la evidencia y `Aspersor` es ancestro de `Pasto`). Por lo tanto, la variable oculta `Aspersor` se elimina sumando (marginalizando) sobre sus estados. Multiplicando los factores y aplicando la evidencia, obtuve que P(Pasto = mojado) = 0.448380. Al normalizar los valores, llegué al resultado de 0.357688 para `Lluvia = sí`. Este cálculo manual coincidió con la aplicación (la diferencia con la enumeración completa fue de $1.1 \cdot 10^{-16}$).

Red de 4 nodos (examen y beca)

Qué modela: Esta red representa los factores que influyen en que un estudiante apruebe un examen y obtenga una beca. Tiene 4 nodos binarios. En la raíz están `Estudia` y `Duerme`, que son independientes entre sí. Ambas variables influyen en el nodo `Aprueba`, y el resultado de aprobar condiciona la probabilidad de que el estudiante reciba la `Beca`. Un detalle estructural importante es que `Aprueba` actúa como un **colisionador** (o convergencia): aunque `Estudia` y `Duerme` son independientes de origen, se vuelven dependientes al momento de condicionar sobre `Aprueba`.

La consulta: La consulta resuelta fue **P(Estudia = sí | Aprueba = sí)**. Es decir, la probabilidad de que el estudiante haya estudiado, sabiendo que aprobó. El resultado obtenido fue **≈ 0.802548** (redondeado a 0.8025).

Cómo lo verifiqué: Para verificarlo, primero apliqué la **poda**: el nodo `Beca` no es ancestro de `Estudia` ni de `Aprueba` y no tiene evidencia; como su tabla de probabilidad suma 1, se puede omitir de los cálculos sin alterar el resultado. Tras la poda, el cálculo se reduce a tres nodos: `Estudia`, `Duerme` y `Aprueba`.

Debido a que `Aprueba` es un colisionador, tuve que sumar (marginalizar) sobre `Duerme` para calcular la probabilidad de que el estudiante apruebe. Obtuve un paso intermedio clave:

P(Aprueba = sí) = 0.628. Finalmente, apliqué el teorema de Bayes para obtener P(Estudia = sí | Aprueba = sí) = 0.802548. El cálculo manual coincidió con el resultado de la aplicación.

Red de 5 nodos (contagio, gripe y faltas)

Qué modela: Esta red representa cómo un contagio influye en la probabilidad de que una persona tenga gripe, y cómo esta, a su vez, influye en la probabilidad de tener fiebre y tos. Finalmente, la fiebre influye en si la persona falta a clases o no. Tiene 5 nodos binarios y es una estructura de cadena con una bifurcación: `Contagio` es padre de `Gripe`; `Gripe` es padre de `Fiebre` y de `Tos` (esta es la bifurcación); y `Fiebre` es padre de `Falta`. No hay colisionadores (nodos con dos padres).

La consulta: La consulta resuelta fue **P(Gripe = sí | Falta = sí)**. Es decir, la probabilidad de que la persona tenga gripe sabiendo que faltó a clases. El resultado obtenido fue **0.609260**.

Cómo lo verifiqué: Para verificarlo, apliqué la eliminación de variables. Aquí sí hubo poda: el nodo `Tos` es una hoja que no es ancestro de la variable de consulta (`Gripe`) ni de la evidencia (`Falta`), y como su tabla de probabilidad suma 1, se puede omitir del cálculo sin alterar el resultado final.

Tras eliminar `Tos`, el cálculo se redujo a sumar (marginalizar) sobre `Contagio` y sobre `Fiebre`. Primero calculé la probabilidad de que la persona tenga gripe (P(Gripe = sí) = 0.31) y luego la probabilidad de que falte (P(Falta = sí) = 0.3002). Al aplicar el teorema de Bayes con estos pasos intermedios, obtuve el resultado de 0.609260, el cual coincide con el valor arrojado por la aplicación.

Red de 6 nodos (tareas y beca)

Qué modela: Esta red representa cómo la dificultad de una tarea y las horas de estudio influyen en la calificación, y cómo esta última influye en la obtención de una beca. Tiene 6 nodos. La estructura es la siguiente: `Dificultad` es el único nodo raíz (y tiene 3 estados: baja, media y alta). De `Dificultad` depende `Horas` (que es binario). A su vez, `Horas` influye en `Cansancio`, en `Entrega` y en `Calificacion`. Además, `Dificultad` también influye directamente en `Calificacion` (que tiene dos padres, `Dificultad` y `Horas`, que además están conectados entre sí). Finalmente, `Calificacion` influye en `Beca`.

La consulta: La consulta resuelta fue **P(Dificultad = alta | Calificacion = buena)**. Es decir, la probabilidad de que la tarea haya sido de dificultad alta sabiendo que el estudiante obtuvo una buena calificación. El resultado obtenido fue **0.061789**.

Cómo lo verifiqué: Para verificarlo, apliqué la eliminación de variables. Primero podé los nodos `Cansancio`, `Entrega` y `Beca`: al ser nodos hoja que no son ancestros de la consulta ni de la evidencia, y como sus tablas de probabilidad suman 1, se pueden omitir sin alterar el resultado.

Tras la poda, el cálculo se redujo a los nodos `Dificultad`, `Horas` y `Calificacion`. Para resolverlo, sumé (marginalicé) sobre `Horas`, pero usando la probabilidad condicional P(Horas | Dificultad), ya que `Horas` depende de `Dificultad`. Con eso obtuve las probabilidades de una calificación buena para cada nivel de dificultad (0.84 para baja, 0.65 para media y 0.19 para alta), y la probabilidad total de una calificación buena (0.615). Finalmente, apliqué el teorema de Bayes y obtuve 0.061789, el cual coincide con el resultado de la aplicación.

Red de 10 nodos (fallas de una PC)

Qué modela: Esta red representa las causas y los síntomas de que una computadora personal falle. Tiene exactamente 10 nodos organizados como un grafo dirigido acíclico. En la raíz está `EnergiaElectrica`, de la cual dependen la `FuenteAlimentacion` y el `UPS`. La fuente alimenta a la `Motherboard` y al `DiscoDuro`. Un caso interesante ocurre con `Motherboard`: es el padre

compartido de CPU y RAM, por lo que ambas comparten una causa en común. Esta estructura no es un árbol estricto, porque CPU y RAM vuelven a unirse más adelante en el nodo PCEnciende (que tiene dos padres), formando un rombo. El DiscoDuro condiciona la probabilidad de que el SistemaOperativo arranque; la RAM influye en la PantallaAzul; y el nodo final, PCEnciende, depende de CPU y RAM.

La consulta: La consulta elegida para verificar el motor de inferencia es **P(PCEnciende = Sí | RAM = Falla)**. Es decir, la probabilidad de que la computadora encienda sabiendo que la memoria RAM falla. El resultado esperado por la aplicación es **0.303920**.

Cómo lo verifiqué: Para comprobar el algoritmo de eliminación de variables, realicé el cálculo a mano. Primero apliqué la **poda**: ignoré los nodos UPS, DiscoDuro, SistemaOperativo y PantallaAzul. La razón matemática es que sus tablas de probabilidad suman 1 en cada fila, por lo que al sumarlos (marginalizarlos) se eliminan de la ecuación sin alterar el resultado final. Tras la poda, el subgrafo relevante queda formado por seis nodos: EnergiaElectrica, FuenteAlimentacion, Motherboard, CPU, RAM y PCEnciende.

Un punto clave es que, como CPU y RAM comparten el padre Motherboard, **no son independientes**. Por lo tanto, al condicionar la consulta en RAM=Falla, es un error usar la probabilidad marginal P(CPU); hay que calcular P(CPU | RAM=Falla). De hecho, mi primer cálculo manual usó P(CPU) en lugar de P(CPU | RAM=Falla) y además una fila equivocada de la tabla de PCEnciende, y arrojó 0.5905, un resultado incorrecto. Al corregirlo y condicionar adecuadamente, obtuve los números intermedios correctos: P(evidencia) = 0.107841 y P(CPU=OK | RAM=Falla) ≈ 0.4617, lo que finalmente lleva al resultado de ≈ 0.3039.

Este cálculo manual coincide con la respuesta de la aplicación (0.303920). Esta coincidencia, junto con las 47 pruebas automatizadas y la comparación contra el método de enumeración completa, da confianza en que el motor de inferencia está implementado correctamente.

Pruebas, resultados, limitaciones y posibles mejoras

> Esta sección forma parte de la memoria técnica del proyecto «Redes bayesianas: editor y consultas con eliminación de variables». Documenta la estrategia de validación del motor descrito en la sección «Explicación del algoritmo implementado» y de la aplicación en su conjunto, los resultados obtenidos, las limitaciones conocidas y las mejoras posibles. La teoría del método se expone en la sección «Fundamentos de redes bayesianas y eliminación de variables».

1. Estrategia de validación

La corrección matemática del método de eliminación de variables es el requisito central del proyecto. Para verificarla se combinaron tres niveles de evidencia independientes entre sí:

1. **Verificación a mano de los cinco ejemplos** (3, 4, 5, 6 y 10 nodos): cada red tiene una consulta resuelta con lápiz y papel, documentada en docs/verificacion-ejemplo01.md … docs/verificacion-ejemplo05.md, y contrastada contra la salida de la aplicación.
2. **Comparación automática contra enumeración completa**: la suite ejecuta BN.infer y BN.enumerate (método independiente descrito en la sección del algoritmo) sobre redes aleatorias y exige que la diferencia máxima sea menor a 10^{-9}.
3. **Prueba de interfaz en navegador real** (tests/ui-smoke.py), que automatiza un navegador Chromium para comprobar el flujo completo: cargar, dibujar el grafo, ejecutar una consulta y ver su verificación.

Las pruebas del motor son autónomas (sin dependencias) y se ejecutan con:

```
npm test          # equivalente a: node tests/test.js
```

La prueba de interfaz es opcional y requiere Playwright:

```
pip install playwright && playwright install chromium
python3 tests/ui-smoke.py
```

2. Pruebas del motor (`tests/test.js`)

La suite está organizada en bloques que cubren: el ejemplo 1 (9 pruebas), errores de consulta (5), validación de redes (14), comparación en redes aleatorias (2), ejemplos 2 a 5 (15) y ejemplos del repositorio (2). En total **47 pruebas, todas correctas (0 con error)**.

2.1. Ejemplo 1 (3 nodos, lluvia–aspersor–pasto)

Además de validar la red, se verifican marginales (`P(Aspersor = encendido) = 0.322`, `P(Pasto = mojado) = 0.44838`), condicionales (`P(Lluvia = sí | Pasto = mojado) = 0.16038 / 0.44838`), una consulta conjunta, la **poda** (`Pasto` es irrelevante para `P(Aspersor | Lluvia = no)` y debe aparecer en `prunedNodes`), el respeto del orden de eliminación indicado por el usuario y la presencia de una **traza** por cada paso.

2.2. Errores de consulta

Se comprueba que la aplicación rechace con mensajes claros: consulta vacía, variable inexistente, variable a la vez consulta y evidencia, estado de evidencia inválido y evidencia con probabilidad 0 (caso en el que condicionar es una división entre cero).

2.3. Validación de redes

Catorce pruebas ejercitan el validador (`BN.validateNetwork`) con casos que deben rechazarse: menos de 3 nodos, más de 10, identificador repetido, padre inexistente, ciclo (`Lluvia ← Pasto`), autociclo, fila de CPT que suma 1.1, probabilidad fuera de `[0, 1]`, CPT con filas faltantes, estado desconocido en `given`, nodo con un solo estado y JSON mal formado. También se comprueban dos casos positivos: **guardar y volver a abrir no pierde información** (los nodos re-serializados son idénticos) y el **orden topológico** (padres antes que hijos).

2.4. Comparación VE contra enumeración en redes aleatorias

Se generan **300 redes aleatorias** con una semilla fija (generador `mulberry32`, semilla 2026): de 3 a 10 nodos, 2–3 estados por variable, 1–2 variables de consulta y 0–3 de evidencia, con ceros ocasionales en las CPT. Para cada una se compara `BN.infer` contra `BN.enumerate`:

- **289 comparaciones válidas** (en 11 casos la evidencia tenía probabilidad 0 y ambos métodos la rechazaron correctamente);
- **diferencia máxima: 4.00×10^{-15}** (el umbral de aceptación es 10^{-9});
- se verifica además que toda distribución resultante suma 1, y que ejecutar con y sin poda produce el mismo resultado.

2.5. Ejemplos 2 a 5 y ejemplos del repositorio

Los bloques finales reproducen en forma de aserciones las verificaciones a mano de los ejemplos 2 a 5 (valores intermedios incluidos) y una prueba que valida y consulta **todos** los `.json` de `ejemplos/`, además de comprobar que `src/ejemplos.generated.js` esté actualizado.

3. Resultados de los cinco ejemplos

Cada red tiene un número distinto de nodos, una consulta resuelta con eliminación de variables, su `P(evidencia)` y su verificación a mano. Los valores mostrados por la aplicación coinciden con los cálculos manuales.

#	Nodos	Red	Consulta	Resultado	P(evidencia)	Verificación
1	3	Lluvia, aspersor y pasto	P(Lluvia = sí | Pasto = mojado)	0.357688	0.448380	docs/verificacion-ejemplo01.md
2	4	Examen y beca	P(Estudia = sí | Aprueba = sí)	0.802548	0.628000	docs/verificacion-ejemplo02.md
3	5	Contagio, gripe y faltas	P(Gripe = sí | Falta = sí)	0.609260	0.300200	docs/verificacion-ejemplo03.md
4	6	Tareas y beca (una variable de 3 estados)	P(Dificultad = alta | Calificacion = buena)	0.061789	0.615000	docs/verificacion-ejemplo04.md
5	10	Fallas de una PC	P(PCEnciende = Sí | RAM = Falla)	0.303920	0.1078405	docs/verificacion-ejemplo05.md

Las capturas de cada consulta ejecutada en la aplicación están en `docs/evidencias/`.

4. Prueba de interfaz (`tests/ui-smoke.py`)

La prueba automatizada abre `index.html` en Chromium y comprueba, sin intervención manual: que Cytoscape.js esté cargado y el grafo se dibuje; que al marcar `Lluvia` como consulta y `Pasto = mojado` como evidencia el resultado muestre `0.357688` y `P(e) = 0.448380` con la verificación «✓» frente a la enumeración; que una consulta sin variables muestre el mensaje de error correspondiente; que una red inválida desde el editor (`{"nodes":[]}`) se rechace con «entre 3 y 10», y que no ocurran errores de JavaScript en la página.

Según el `README.md`, además de esta prueba automatizada en Linux/Chromium, la interfaz se probó manualmente en Linux (Firefox), donde se cargó el ejemplo 5 y se ejecutó su consulta, y se comprobó que la versión en línea carga y se muestra sin errores en Windows (Brave/Chrome), macOS (Safari/Chrome) y Android (Chrome).

5. Limitaciones

A la vista de lo anterior, las limitaciones conocidas del sistema son:

- **Coste exponencial en el peor caso.** VE es exacto, pero su coste depende del orden de eliminación y puede crecer de forma exponencial con la anchura de árbol de la red. El límite de 10 nodos acota la red, pero **no** el número de estados por nodo; una red con variables de muchos estados (o muy densa) puede generar factores intermedios grandes y agotar memoria o tiempo.
- **Heurística de orden subóptima.** El criterio greedy de mínimo tamaño es barato y suficiente para las redes del proyecto, pero no garantiza el orden óptimo; existen heurísticas mejores (p. ej., *min-fill*).
- **Solo inferencia exacta y evidencia «dura».** No hay inferencia aproximada ni se admite evidencia blanda (*soft evidence*); la evidencia fija un único estado por variable.
- **Variables únicamente discretas y finitas.** No se modelan variables continuas ni gaussianas.

- **Sin compilación reutilizable.** Cada consulta se resuelve desde cero; no se compila la red (p. ej., un árbol de unión o circuito aritmético) para amortizar el coste entre consultas sucesivas.
- **La comparación con enumeración es opcional y costosa.** Al activarla, la enumeración completa es exponencial en el número de variables; en redes densas o con muchos estados puede resultar lenta, aunque para los casos de prueba (hasta 10 nodos) es manejable.
- **Precisión de punto flotante.** Los resultados usan doble precisión; las diferencias observadas frente a la referencia son del orden de 10^{-15}, insignificantes para la interpretación, pero inherentes a la aritmética flotante.

6. Posibles mejoras

- **Árbol de unión (junction tree / clique tree).** Compilar la red una sola vez y responder múltiples consultas reutilizando la estructura de cliques, lo que reduce el coste amortizado.
- **Mejores heurísticas de orden de eliminación** (*min-fill*, *min-weight*, búsqueda con poda) para reducir el tamaño de los factores intermedios en redes densas.
- **Evidencia blanda y variables continuas o híbridas**, ampliando la expresividad del modelo.
- **Compilación a circuito aritmético** o caché de factores para consultas repetidas con distinta evidencia.
- **Cobertura de pruebas adicional:** variables con 4 o más estados, redes más densas y pruebas basadas en propiedades (*property-based testing*).
- **Interoperabilidad:** importar/exportar otros formatos estándar (BIF, XDSL) y exportar el reporte de resultados a PDF o CSV.

Conclusiones

- Se construyó una aplicación web que permite crear, validar, guardar y consultar redes bayesianas de 3 a 10 nodos con eliminación de variables, sin servidor.
- Las cinco redes de ejemplo (3, 4, 5, 6 y 10 nodos) tienen una consulta resuelta con la aplicación y verificada a mano; en todos los casos el resultado de la aplicación coincide con el cálculo manual y con la enumeración completa.
- Las pruebas automáticas incluyen las verificaciones de los ejemplos, los errores de validación y de consulta, y redes aleatorias de 3 a 10 nodos en las que la eliminación de variables coincide con la enumeración.
- Resolver las redes a mano mostró dos ideas importantes: la poda se justifica porque las tablas de los nodos descartados suman 1, y las variables que comparten un padre no son independientes al condicionar sobre una de ellas.
- Las limitaciones (coste exponencial en el peor caso, orden de eliminación heurístico, evidencia solo exacta y variables discretas) y las posibles mejoras se describen en la sección de pruebas.

Referencias

- Koller, D. y Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann.
- Zhang, N. L. y Poole, D. (1994). A simple approach to Bayesian network computations. *Proceedings of the Canadian Conference on AI*.
- Franz, M. et al. (2016). Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2), 309–311.

Contribuciones y declaración de herramientas

Quién hizo qué

Integrante	Contribución
Sebastián Rendon Silva	Diseño de las redes de 4, 5, 6 y 10 nodos, sus verificaciones a mano, capturas de evidencia y pruebas; formato de datos; README; repositorio y ensamblado de documentos.
Jesús Rodríguez	Formularios para crear, renombrar y eliminar nodos, exportación del grafo a PNG, historial de deshacer y rehacer, manual de usuario y documentación de la interfaz.
Uriel Guzman Cardoso	Secciones de esta memoria sobre los fundamentos de redes bayesianas y eliminación de variables, la explicación del algoritmo implementado, y las pruebas, resultados, limitaciones y mejoras (`docs/memoria-fundamentos.md`, `docs/memoria-algoritmo.md` y `docs/memoria-pruebas.md`).

El historial de commits de GitHub es la fuente detallada de la contribución de cada integrante.

Declaración de herramientas

- **Asistencia de IA (Claude, de Anthropic).** El esqueleto inicial del repositorio (motor de eliminación de variables, validaciones, interfaz mínima, pruebas y plan de trabajo) se elaboró con ayuda de Claude, y el equipo lo revisó. Los borradores de las secciones de introducción, herramientas y arquitectura, conclusiones y referencias de esta memoria, y el ensamblado de este documento, también se prepararon con asistencia de Claude y deben ser revisados por el equipo.
- Las variables, los estados, las probabilidades y los cálculos a mano de las redes de ejemplo de Sebastián Rendon Silva son de su autoría, y Claude se usó para revisar y dar formato.
- **Bibliotecas:** Cytoscape.js (MIT).
- **Repositorio:** https://github.com/2201248e-1170/RedesBayesianas-App · **Versión en línea:** https://2201248e-1170.github.io/RedesBayesianas-App/

Anexo

Módulo de interfaz y pruebas

Documentación técnica de Jesús Rodríguez

Documentación Técnica: Módulo de Interfaz y Pruebas

Esta sección detalla los módulos implementados en `app.js` para la gestión de la red bayesiana, abarcando la exportación visual, el control de estados y la manipulación segura de la estructura de nodos.

1. Exportación de Grafo a PNG

Permite al usuario descargar el estado actual del diagrama visual en una imagen de alta resolución con fondo sólido.

JavaScript

```
C/C++
$('btn-exportar-img').addEventListener('click', () => {
    // cy.png() toma la foto directamente del lienzo con fondo blanco
    const pngUrl = cy.png({ bg: '#ffffff', full: true });

    // Crear un enlace invisible para forzar la descarga nativa
    const link = document.createElement('a');
    link.href = pngUrl;
    link.download = 'grafo-red-bayesiana.png';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
});
```

- **¿Qué hace?** Genera un archivo `.png` del lienzo de Cytoscape y lo descarga al equipo del usuario.
- **¿Cómo lo hace?** Utiliza la API nativa de Cytoscape (`cy.png`) para renderizar el estado actual del canvas. Como los navegadores no permiten forzar descargas directamente por seguridad, se crea un elemento `<a>` temporal, se le asigna la imagen como fuente en formato base64 (`href`), se simula un clic programático (`link.click()`) y se destruye el elemento para no ensuciar el DOM.

2. Sistema de Historial (Deshacer / Rehacer)

Implementa una máquina del tiempo en memoria para revertir o recuperar acciones estructurales en el lienzo mediante atajos de teclado estándar.

JavaScript

```C/C++
let historialRed = [];
let pasoActual = -1;

function guardarEstadoRed() {
    const estado = $('editor').value;
    if (pasoActual === -1 || historialRed[pasoActual] !== estado) {
        historialRed = historialRed.slice(0, pasoActual + 1);
        historialRed.push(estado);
        pasoActual++;
    }
}

// Guardar el estado inicial apenas cargue la página y al soltar el ratón
setTimeout(guardarEstadoRed, 500);
document.addEventListener('mouseup', () => {
    setTimeout(guardarEstadoRed, 100);
});
$('btn-aplicar').addEventListener('click', guardarEstadoRed);

// Detectar atajos de teclado (Ctrl+Z / Ctrl+Y)
document.addEventListener('keydown', (e) => {
    if (e.ctrlKey && e.key.toLowerCase() === 'z') {
        e.preventDefault();
        if (pasoActual > 0) {
            pasoActual--;
            loadText(historialRed[pasoActual]);
        }
    }
    if (e.ctrlKey && e.key.toLowerCase() === 'y') {
        e.preventDefault();
        if (pasoActual < historialRed.length - 1) {
            pasoActual++;
            loadText(historialRed[pasoActual]);
        }
    }
});
```

- **¿Qué hace?** Permite usar `Ctrl+Z` para deshacer y `Ctrl+Y` para rehacer cualquier cambio en la red bayesiana.
- **¿Cómo lo hace?** Mantiene un arreglo (`historialRed`) que guarda copias exactas del JSON (que es la "verdad absoluta" de la red) cada vez que ocurre una acción significativa (clics, botón aplicar). La variable `pasoActual` actúa como un puntero. Al presionar `Ctrl+Z`, el puntero retrocede una posición y manda ese texto JSON a la función `loadText()`, la cual redibuja el lienzo. Si se hace un cambio nuevo estando atrás en el historial, la función `slice` recorta el futuro alternativo para evitar conflictos de línea temporal.

3. Generación Dinámica de Tablas de Probabilidad (CPT)

Calcula el producto cartesiano de los estados de los nodos padres para construir una tabla matemática estrictamente válida para el motor bayesiano.

JavaScript

```
C/C++
function buildDefaultCPT(node, allNodes) {
       const parentNodes = node.parents.map(pId => allNodes.find(n => n.id === pId)).filter(Boolean);
    let combinations = [{}];

    // Producto cartesiano de los estados de todos los padres
    for (const p of parentNodes) {
        const newCombos = [];
        for (const c of combinations) {
            for (const state of p.states) {
                newCombos.push({ ...c, [p.id]: state });
            }
        }
        combinations = newCombos;
    }

    // Probabilidad equitativa por defecto
    const prob = Number((1 / node.states.length).toFixed(4));
    const probs = {};
    node.states.forEach(s => probs[s] = prob);

    return combinations.map(c => ({
        given: c,
        probs: { ...probs }
    }));
}
```

- **¿Qué hace?** Genera la estructura exacta de objetos `cpt: [{given: {}, probs: {}}]` que exige el validador estricto de la aplicación cada vez que se crea o edita un nodo.
- **¿Cómo lo hace?** Itera sobre los nodos padres para multiplicar sus estados (ej. si el padre tiene 2 estados y el hijo 2, genera 4 filas). Asigna a todas las combinaciones una probabilidad equitativa inicial (`1 / número de estados`). Esto previene la excepción de "fila no es un objeto" y evita la destrucción de las aristas en el grafo.

4. Delegación de Eventos para Formularios de Nodos (Crear/Editar/Eliminar)

Gestiona la interfaz de los paneles inferiores asegurando sincronización total entre el lienzo, el JSON subyacente y las dependencias de parentesco.

JavaScript

```
C/C++
let nodoSeleccionadoId = null;

// Cargar datos automáticamente al tocar el lienzo
document.addEventListener('mouseup', () => {
    setTimeout(() => {
        try {
            if (typeof cy !== 'undefined' && cy.$) {
                const sel = cy.$('node:selected');
                if (sel.length > 0) {
                    const id = sel[0].id();
                    const elEditor = document.getElementById('editor');
                    if (elEditor && elEditor.value) {
                        const data = JSON.parse(elEditor.value);
                        const nodo = data.nodes.find(n => n.id === id);
                        if (nodo) {
                            nodoSeleccionadoId = id;
                            document.getElementById('nodo-id').value = id;
                                    document.getElementById('nodo-estados').value =
nodo.states.join(', ');
                        }
                    }
                }
            }
        } catch(e) {}
    }, 150);
});

// Radar global para botones Guardar y Eliminar
document.addEventListener('click', (e) => {
    if (e.target && e.target.id === 'btn-guardar-nodo') {
```

```
        // ... (extracción de valores del DOM) ...
        const data = JSON.parse(elEditor.value);
        let idViejo = nodoSeleccionadoId || 'Aspersor';
        let nodeIndex = data.nodes.findIndex(n => n.id === idViejo);

        if (nodeIndex !== -1 && idViejo !== nuevoId) {
            data.nodes[nodeIndex].id = nuevoId;
            data.nodes.forEach(n => {
                    if (n.parents) n.parents = n.parents.map(p => p === idViejo ?
nuevoId : p);
            });
            // Migrar posiciones
        } else if (nodeIndex === -1) {
              data.nodes.push({ id: nuevoId, type: "discrete", states: [], parents:
[], cpt: [] });
            nodeIndex = data.nodes.length - 1;
        }

        let targetNode = data.nodes[nodeIndex];
        targetNode.states = estados;
        targetNode.cpt = buildDefaultCPT(targetNode, data.nodes);

        // Reconstruir CPT de TODOS los nodos hijos
        data.nodes.forEach(n => {
            if (n.parents && n.parents.includes(nuevoId)) {
                n.cpt = buildDefaultCPT(n, data.nodes);
            }
        });

        elEditor.value = JSON.stringify(data, null, 2);
        nodoSeleccionadoId = nuevoId;
        if (typeof loadText === 'function') loadText(elEditor.value);
    }

    if (e.target && e.target.id === 'btn-eliminar-nodo') {
        // ... (búsqueda del nodo por ID) ...
        if (nodeIndex !== -1) {
            data.nodes.splice(nodeIndex, 1);
            data.nodes.forEach(n => {
                if (n.parents && n.parents.includes(id)) {
                    n.parents = n.parents.filter(p => p !== id);
                     n.cpt = buildDefaultCPT(n, data.nodes); // Reconstruir tabla sin
ese padre
                }
            });
            // ... (Actualización de DOM y JSON) ...
        }
    }
});
```

- **¿Qué hace?** Autocompleta los campos al seleccionar un nodo. Permite renombrar o crear variables y eliminarlas sin dejar conexiones huérfanas o errores estructurales.
- **¿Cómo lo hace?**

  Autocompletado: Un retraso `setTimeout` de 150ms espera a que Cytoscape registre la selección nativa, extrae el ID, busca el objeto en el JSON y lo inyecta en los `input`.

  Delegación global: En lugar de asignar un `.addEventListener` directo a los botones (que pueden perder la referencia si el DOM se recarga), el `document.addEventListener('click')` intercepta todos los clics de la página y filtra por la ID del objetivo (el "radar de clics").

  Migración segura: Al renombrar, escanea el arreglo `parents` de todos los demás nodos y actualiza el nombre viejo por el nuevo. Al eliminar, borra las referencias del arreglo `parents` en los nodos dependientes y vuelve a ejecutar `buildDefaultCPT` en cada uno para sanear las tablas de probabilidad al instante.